

multiple spanning Tree / MST are possible.

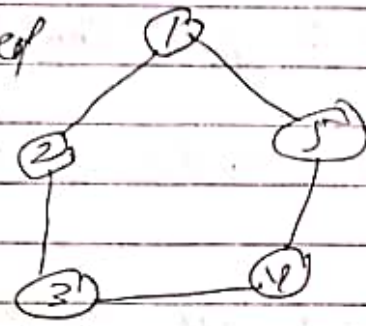
Page No.
 Date 11/05/21

doesn't contain cycle

← MST (minimum spanning Tree)

should be connected

non-weighted graph →



$G(V, E)$

↓

$G'(V', E')$

Note: $V' = V$

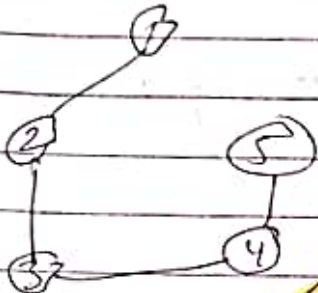
$E' \subset E$

$|E'| = |V| - 1$

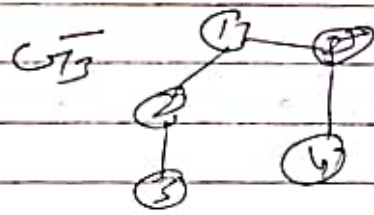
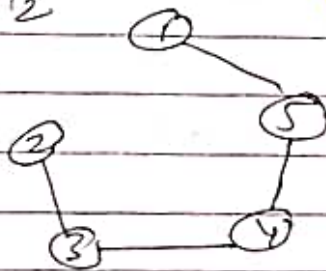
in G' graph

Tree = Acyclic connected graph
(not) cycle

G_1

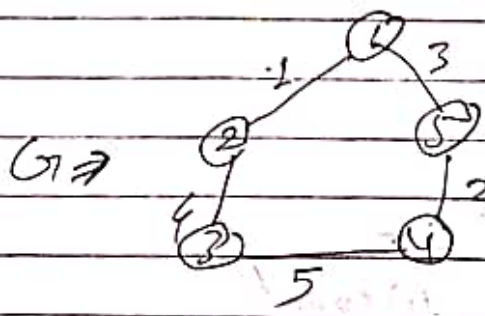


G_2



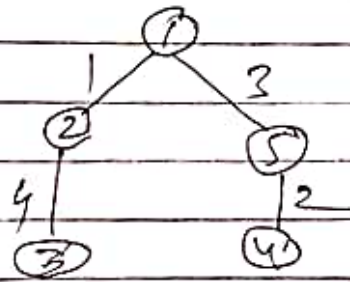
→ G_1, G_2, G_3 are spanning Tree of G

Now, MST will be on weighted graph



$\Rightarrow$ MST(G) →

Cost = 10



Properties of ST & MST Distinct (Weights) = Unique MST

Page No. _____

Date: ____/____/____

→ From complete graph by remove $\max(e-n+1)$ edge we construct a ST.

→ Disconnected graph doesn't have any ST.

→ Add any single edge make ST a cycle (loop)

→ Remove any single edge from ST with disconnect graph

(ST) → Spanning Tree ~~is~~ not contains any cycle

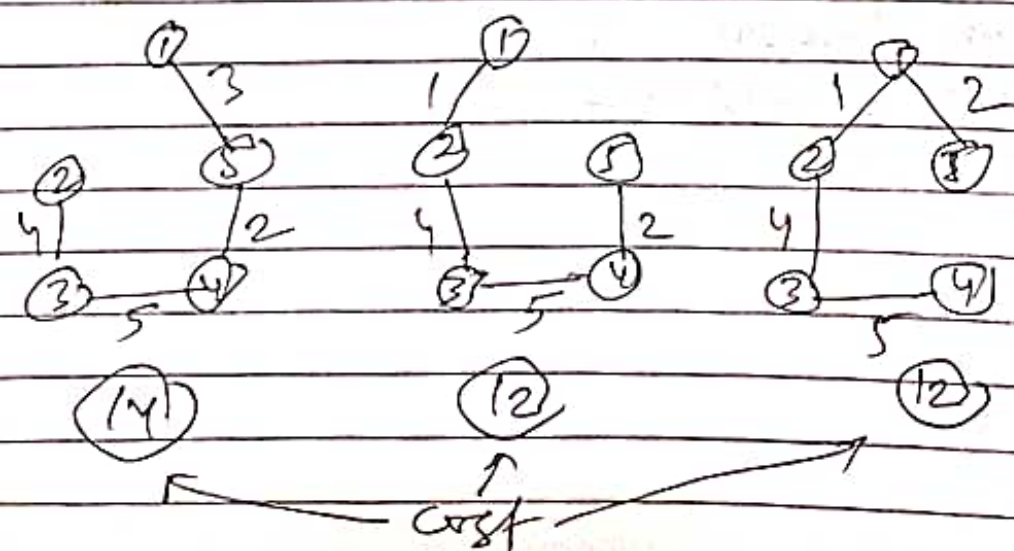
→ Spanning Tree doesn't disconnected (ST)

→ If each weight is distinct in ST then it is/has unique MST

→ A complete undirected graph have (2^{n-2}) num of ST.

→ A graph is conn't & undirected then it has atleast one ST.

Spanning Tree of weighted G



→ A Graph has multiple spanning trees

Note $\Rightarrow$ leaf node = only one edge it conn^{ct}

Page No. _____

Date: ____/____/____

Proof

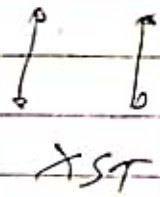
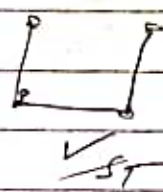
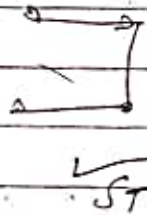
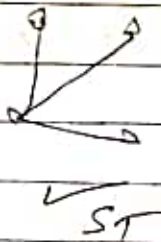


$$n = 4$$

$$e = 6$$

max $(e - n + 1)$ Remove

$$\text{max } (6 - 4 + 1) = 3 \text{ edges remove}$$



$\Rightarrow$ Violate max edge Remove

$$\text{max}(S_4) = 4^{4-2} = 4^2 = 16$$

Edges = 4 removed

$\therefore X_{S_4}$ discarded

prog = DS + Algo $\rightarrow$ well define Procedure to solve specific Problem

$\downarrow$
A way of storing & organizing data, so it can be used effectively

Property
Ex $\rightarrow$

A complete undirected graph have (n^{n-2}) many ST.

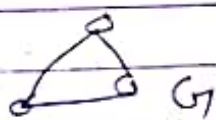
Sol $\rightarrow$ $n=4$ in complete graph,

Proof $\Rightarrow \therefore \text{num}(ST) = 4^{4-2} = 4^2 = \underline{16}$

Graph with 4 vertex has 16 spanning Trees.

Property $\Rightarrow$ From complete graph by removing $\text{max}(e-n+1)$ edges, we can construct a ST.

Proof $\Rightarrow$

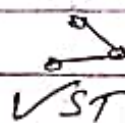
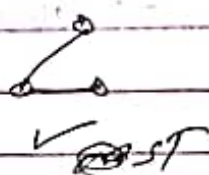


$e=3$
 $n=3$

$\therefore \text{Max}(e-n+1)$ Remove

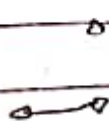
$\text{max}(3-3+1)$

$\text{max}(1)$ Remove



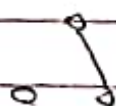
want to
if Remove $> \text{max}(\text{Remove})$ then $G \neq ST$.

Remove (2 E)



X ST

discont



X ST

discont

~~the Basics~~

Standard Greedy Algo

- ① Job sequencing problem
- ② Huffman coding
- ③ policemen catch thieves
- NP-complete problem
- ④ Graph coloring
- ~~⑤ Penalty problem~~
- ⑥ Bin packing

Greedy Algo for graph

- ① Kruskal's mst
- ② Prim's mst
- ③ Boruvka's mst
- ④ Dijkstra's shortest path
- ⑤ Correctness of greedy Alg
- ⑥ minimum cost to connect all cities
- ⑦ max flow problem

Greedy Algo in Array

- min/max product of subarray
- All array ops problems

Greedy algo in OS

- partitioned allocation methods (first/worst/best fit)
- SJF, scheduling priority task in limited time
- page Replacement policies

Greedy

→ Subset sum problem
→ longest common subsequence
Dynamic

① make decision considering the first stage

① make decision at every stage

② less efficient
no guarantee of optimal solution

② more efficient & guaranteed of optimal solution

③ Efficient in term of memory

③ not efficient for mem as reqs more memory & dp table

④ first makes a choice that looks best at the time & then solves a resulting subproblem

④ solve all subproblem & then select one that helps to find the optimal solution

It is
⑤ An Algo that follow the problem-solving heuristic of making the locally optimal choice at each stage with the intent of finding a global optimum.

⑤ It is An Algo that helps to efficiently solve a class of problems that have overlapping subproblems & optimal substructure property.

Ex → optimal substructure property, overlapping subproblems property, Bitmasking & Dynamic Prog, Fibonacci num, Bellman

Worse case
if Adjacency
matrix is
used $O(V^2)$

used with "d-ary" heap = Binary heap

Greedy Approach

* Prim's

Algo

→ Initialises with any Vertex

Random

It resembles Dijkstra's Algo

→ Remove self loop

→ It uses to implt fibonaccihheap

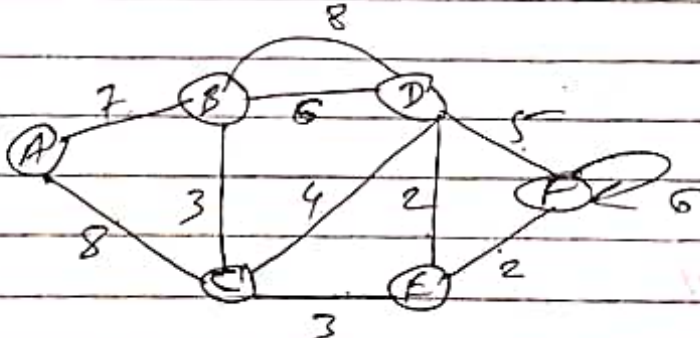
→ Remove Parallel edges (keep only min(edge) among 2)
parallel edges

→ choose any Arbitrary vertex

→ select min(Edge) incident from that vertex
weighted

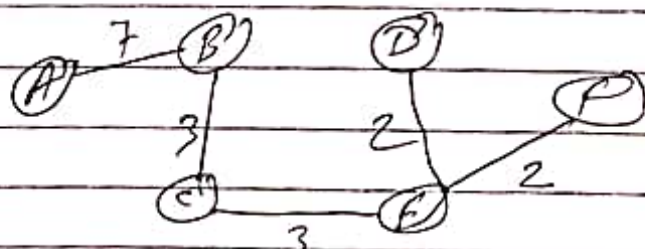
→ while moving ahead, always compare remaining
edges of starting vertex.
(check)

Ex



→ Compare Each vertex's
Edge Cost with every
vertices till reach
last node (vertex).

Soln



→ Max (remove)

$$\text{Max } (e - n + 1) = (9 - 6 + 1) = 4$$

$$\rightarrow \text{MST} = 17$$

$$\text{Cost} =$$

→ MST path = A, B, C, E, D, F

→ Edges = ~~A-B~~, AB, BC, CE, ED, EF

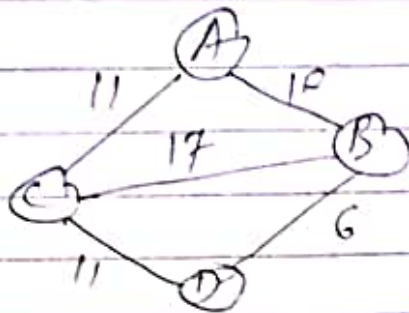
$$\rightarrow \text{num(Edges)} = 5$$

Prim's Algo $\rightarrow$ Best for dense graph

Page No. _____

Date: ____/____/____

Ex $\Rightarrow$

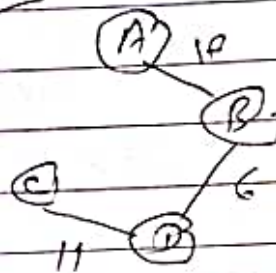


Find MST using Prim's algorithm?

Soln $\Rightarrow$ Pick A, min[Edges] $\rightarrow$ AB selected
 $\left\{ \begin{matrix} AC, AB \\ 11, 10 \end{matrix} \right\}$

from B, min[BC, BD] $\rightarrow$ BD selected
 $\left\{ \begin{matrix} 17, 6 \end{matrix} \right\}$

from D to C selected
 as last edge
 DC selected



MST = 27
cost

In MST $\frac{\text{num(Edges)}}{|E|} = |V| - 1 = 4 - 1 = 3$
 Edges are = {AB, BD, DC}

Remove max Edges
 $(E - V + 1)$
 $= (5 - 4 + 1) = 2$

only At max 2
 edges remove
 to make ST

$\left(\frac{\text{num(Edges)}}{\text{MST}} = \text{num}(E') = |V| - 1 = 6 - 1 = 5 \right)$

Find num (S.T) $\leftarrow$ Kirchhoff

$\leftarrow$ complete graph

Page No.

Date 28/10/23

* Spanning Tree

$\rightarrow$ If graph is not complete then we can use Kirchhoff theorem.

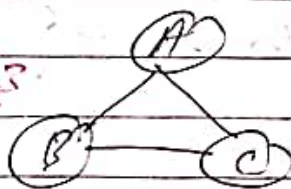
$\rightarrow$ ① Design Adjacency matrix of graph.

$\rightarrow$ ② Diagonal values are replace by degree of that vertex.

$\rightarrow$ ③ All non-diagonal values

$\begin{matrix} \text{1's} \\ \Downarrow \\ \text{Replace By} \\ (-1) \end{matrix} \qquad \begin{matrix} \text{0's} \\ \Downarrow \\ (0) \end{matrix}$

Ex $\Rightarrow$ Find the num of spanning tree using Kirchhoff.



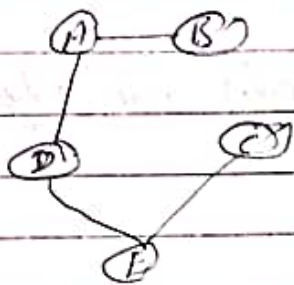
$$\begin{matrix} & A & B & C \\ \begin{matrix} A \\ B \\ C \end{matrix} & \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix} \end{matrix}$$

$$\begin{matrix} & A & B & C \\ \begin{matrix} A \\ B \\ C \end{matrix} & \begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix} \end{matrix}$$

num(S.T) $\Rightarrow$ Laplacian $\Rightarrow$ cofactor $\Rightarrow$ M [any element]

$$\begin{aligned} M[2] &= [2 \times 2 - (-1 \times -1)] \\ &= 4 - 1 = 3 \end{aligned}$$

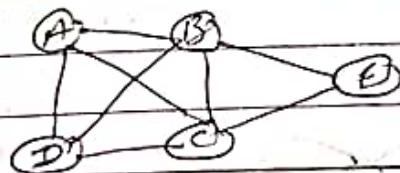
* sparse graph = Graph conn through min num of edges.



$$\text{num(Edges)} = n$$

* Dense graph

⇒ Graph is conn through maximal num of edges



$$\text{num(edges)} \approx n^2$$

$$\therefore \text{num(ET)} = 3^3 - 2 \cdot 3^1 = 8$$

from sorted Edges that Union all vertex

$$\therefore O(E \log V)$$

"Kruskal's MST"

use min heap

→ Remove all self loop

→ Remove all parallel edges (keep only min(Edge) among parallel edges)

→ Arrange All edges in Increasing order

→ start construct in that order only, But ~~not~~ ^{Not Add} edge which form cycle, ignore such edges.

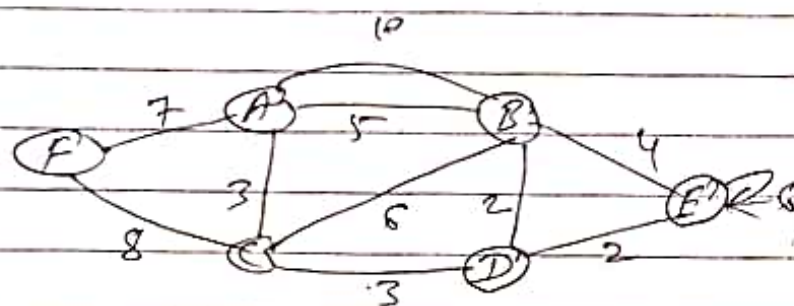
⊗ Kruskal

MST(G) ⇒

$$\text{num}(E') = |V| - 1$$

$$\text{num}(V') = V$$

Ex ⇒



Soln → Remove E's self loop & AB's parallel edge 10 weight.

→ Write All edges in Increasing order of weight

BD = 2 ✓ → start construct with minimum cost.

DE = 2 ✓

AC = 3 ✓

CD = 3 ✓

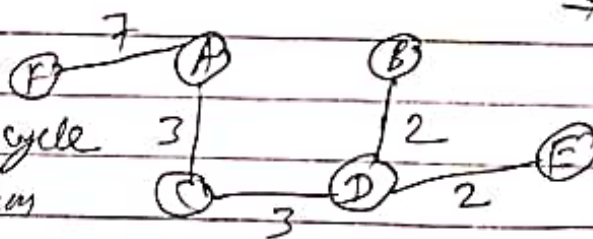
BE = 4 ✗ form cycle

AB = 5 ✗ form cycle

CB = 6 ✗ form cycle

AF = 7 ✓

CF = 8 ✗ form cycle



→ MST cost = 17

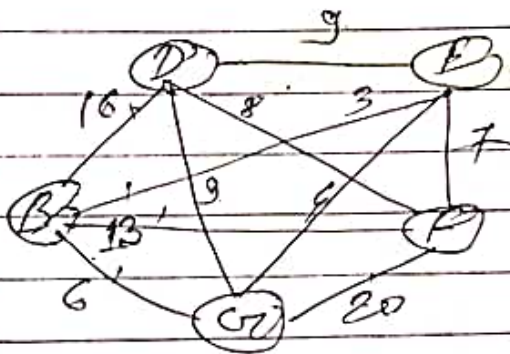
$$\text{num(Edges)} = 5 (V - 1)$$

$$\rightarrow \text{max(Remove)} = (9 - 6 + 1) = 4$$

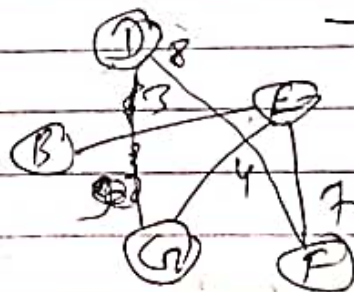
Kruskal's Algo = Implt using Disjoint-set data structure

→ Cost of MST → Kruskal } → should be = minimum
→ Prim's

Ex: → which of the following edges form minimum spanning tree on the graph using Kruskal's algo?



Soln → BE = 3,
GE = 4,
BG = 6 X
EF = 7,
DF = 8,
DG = 9 X,
DE = 9 X
BF = 13 X
BD = 16 X
GF = 20 X



→ Once All vertices
Covered without
cycle, no
need to Add
or check any
Extra Edges.

MST = 22

Cost

Remove

max = 10 - 5 + 1

= 6

|E| = 5 - 1 = 4

Edges → BE, GE, EF, DF

Prim's Algo

Kruskal Algo

- ① It generates the MST starting from the Root vertex.
 - ② select the Root vertex
 - ③ It Run faster in Dense graph
 - ④ It traverse ^{any} one node multiple time in order to get its minimum distance.
 - ⑤ Adjacency matrix, Binary heap, Fibonacci heap is used in Prim's algo.
 - ⑥ Time Complexity $\Rightarrow$
 $\text{Binary heap } = O(E \log V)$
 $\text{Fibonacci heap } = O(E + V \log V)$
- ① It Generates the MST starting from the least weighted Edge. (minimum)
 - ② select the shortest Edge
 - ③ It Run faster in sparse graph
 - ④ It traverse ^{any} edge only once, based on cycle it will either reject or accept it.
 - ⑤ Disjoint set is used in Kruskal Algo
 - ⑥ Time Complexity $= O(E \log V)$

3 types of Edges while constructing MST

- ① Always connⁿ edge (^{mst} min value edge)
- ② Never connⁿ edge (max value edge)
- ③ may or may not be connⁿ edge

Return Boolean Value

Dynamic Approach
Prog

Page No.

Date: 18/05/21

Time = $O(V \cdot E)$

* Bellman Ford Algo

(Single source shortest path)

→ Const & directed Graph

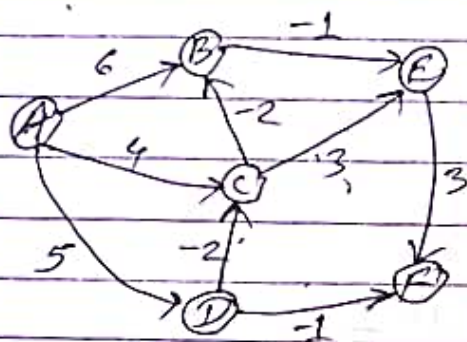
→ Allow Negative Edge weighted

Relaxation Principle
(Iterative method)

working steps

① → Go on Relaxing all the edges $(n-1)$ times:

$G = \text{cost}$



Relaxing $\Rightarrow$ if $d[u] + G(u,v) < d[v]$

Relaxation

update $\Rightarrow d[v] = d[u] + G(u,v)$

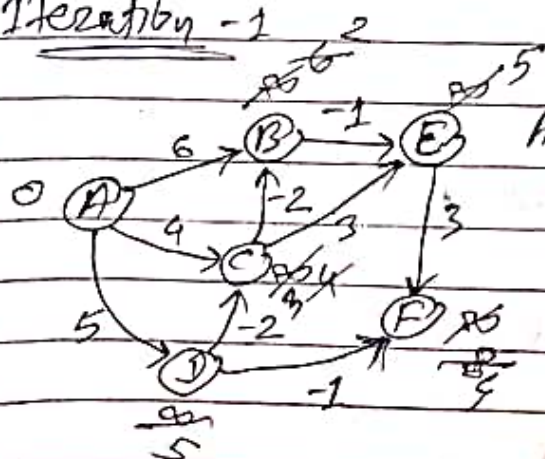
→ An Approximate to the correct distance is replaced progressively by more accurate value till optimal solution

② → Walk down all edges in any order

Edges = (A,B), (B,E), (E,F), (D,F), (A,D), (A,C), (C,B), (C,E), (D,C)

→ Num (Iteration) req^d to find final shortest path = $n-1$

Iteration - 1



$A \rightarrow AB: d(A) + \text{cost}(A,B) < d(B)$

$$0 + 6 < \infty \quad \checkmark$$

$\therefore$ update $d(B) = 6$

$A \rightarrow BE: d(B) + \text{cost}(B,E) < d(E)$

$$6 + (-1) < \infty$$

$$5 < \infty \quad \checkmark$$

$\therefore$ update $d(E) = 5$

~~Ellen~~ $O(V)$ for $(V-1)$ Iteration } $\therefore$ Time Complexity $O(VE)$
~~for~~ for ~~all~~ Edge $O(E)$ } $O(VE)$

Page No. _____

Date: ____/____/____

$$\begin{aligned}
 \underline{A: EF} &\rightarrow d(E) + \text{cost}(E, F) < d(F) \\
 5 + 3 &< \infty \\
 8 &< \infty \quad \therefore \text{update } d(F) = 8
 \end{aligned}$$

$$\begin{aligned}
 \underline{A: AD} &\rightarrow d(A) + \text{cost}(A, D) < d(D) \\
 0 + 5 &< \infty \quad \checkmark \\
 &\therefore \text{update } d(D) = 5
 \end{aligned}$$

$$\begin{aligned}
 \underline{A: DF} &\rightarrow d(D) + \text{cost}(D, F) < d(F) \\
 5 + (-1) &< 8 \\
 4 &< 8 \quad \checkmark \\
 &\therefore \text{update } d(F) = 4
 \end{aligned}$$

$$\begin{aligned}
 \underline{A: AC} &\Rightarrow d(A) + \text{cost}(A, C) < d(C) \\
 0 + 4 &< \infty \quad \checkmark \\
 &\therefore \text{update } d(C) = 4
 \end{aligned}$$

$$\begin{aligned}
 \underline{A: CE} &\Rightarrow d(C) + \text{cost}(C, E) < d(E) \\
 4 + 3 &< 5 \\
 7 &< 5 \quad \times \\
 &\therefore \text{Don't update}
 \end{aligned}$$

$$\begin{aligned}
 \underline{A: CB} &\rightarrow d(C) + \text{cost}(C, B) < d(B) \\
 4 + (-2) &< 6 \\
 2 &< 6 \quad \checkmark \\
 &\text{update } d(B) = 2
 \end{aligned}$$

$$\begin{aligned}
 \underline{A: DC} &\Rightarrow d(D) + \text{cost}(D, C) < d(C) \\
 5 + (-2) &< 4 \\
 3 &< 4 \quad \checkmark \quad \text{update } d(C) = 3
 \end{aligned}$$

Dijkstra's Algo more efficient than
 $T(n) = O(V^2)$ for Adjacency matrix

Bellman Ford

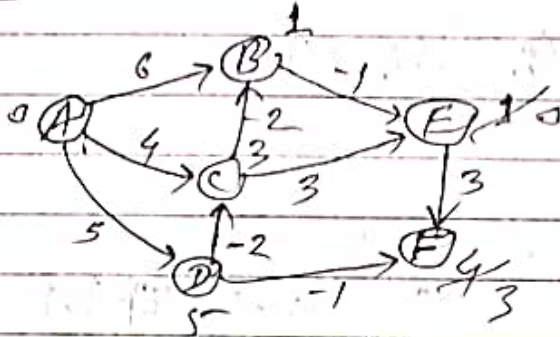
$T(n) = O(VE)$

Date: 11

if $V = E = n$

$\therefore O(V^2)$

Iteration 3



min

weight $\Rightarrow$ shortest distance

A = 0

B = 1

C = 3

D = 5

E = 0

F = 3

+ 12

A: AB $\rightarrow 0 + 6 < 1$ $\times$ don't change

A: BE $\rightarrow 1 + (-1) < 1$

$0 < 1$ $\checkmark$

$\therefore$ change $d(E) = 0$

A: EF $\rightarrow 0 + 3 < 4$ $\checkmark$

$\therefore$ change $d(F) = 3$

A: AC $\rightarrow 0 + 4 < 3$ $\times$ don't change

A: AD $\rightarrow 0 + 5 < 5$ $\times$ don't change

A: DF $\rightarrow 5 - 1 < 3$ $\times$ don't change

A: CB $\rightarrow 3 - 2 < 1$ $\times$ don't change

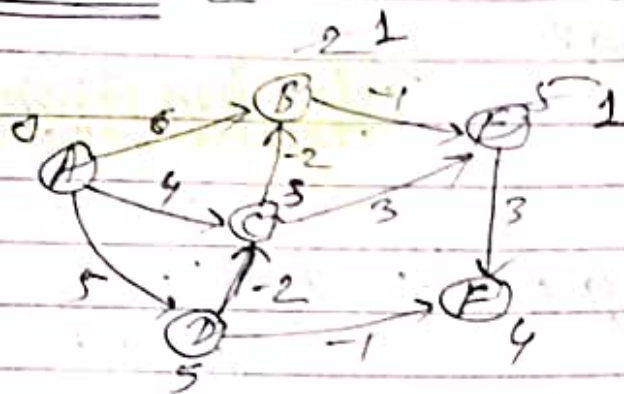
A: CE $\rightarrow 3 + 3 < 0$ $\times$ don't change

A: DC $\rightarrow 5 - 2 < 3$ $\times$ don't change

Iteration 4 &

Iteration 5

will have no
change
states.

Iteration - 2

if relaxing value of
current cost of $d(v)$ are
equal (same), then don't
change.

A: AB $\rightarrow 0 + 6 < 2$ $\times$ don't change

A: BE $\rightarrow 2 + (-1) < 5$
 $1 < 5$ $\checkmark$

$\therefore$ change $d(E) = 1$

A: EF $\rightarrow 1 + 3 < 4$ $\times$ don't change

A: AC $\rightarrow 0 + 4 < 3$ $\times$ don't change

A: AD $\rightarrow 0 + 5 < 5$ $\times$ don't change

A: DF $\rightarrow 5 - 1 < 4$ don't change

A: CB $\rightarrow 3 + (-2) < 2$
 $3 - 2 < 2$
 $1 < 2$ $\checkmark$

$\therefore$ change $d(CB) = 1$

A: CE $\rightarrow 3 + 3 < 1$ $\times$ don't change

A: DC $\rightarrow 5 - 2 < 3$ $\times$ don't change

run on weighted
Directed graph

not apply for (-ve) edges (weight edges)
Negative

Page No. _____
Date: / /

Dijkstra's Single Source

shortest path just find shortest path

Greedy Approach

Not Needed to connect or cover all nodes

single node fixed & shortest path from this node to destination node found by selecting minimum weight.

Data structure used = min Priority Queue

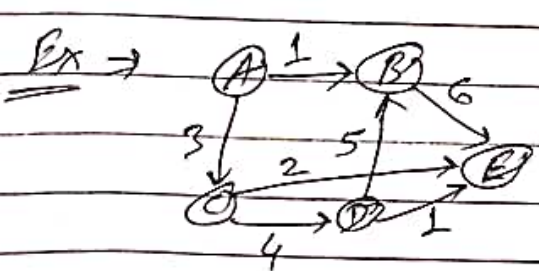
Time complexity = $O(V^2)$

3 Priority Queue
min
open Reg

Running Time Complexity using Binary heap = $O(E \log V)$

Reg & open each
insert
Extract min
Decrease key

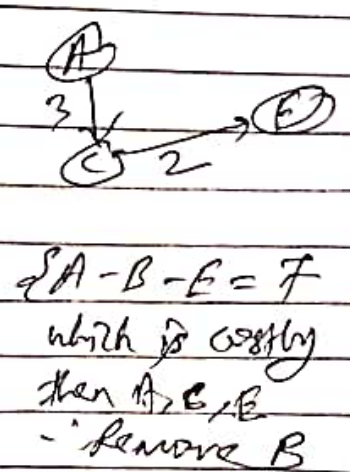
Total min (V) times
that open perform



Find shortest path of A to B source & destination B?

Soln

		A	B	C	D	E
Initially	A	0	∞	∞	∞	∞
{A, B}		0	1	3	∞	∞
{A, B, C}		0	1	3	∞	7
∴ {A, C, E}		0	1	3	7	5
{A, C, E}						5



(-) cycle = sum of all edges forming cycle = Negative

Page No. _____

Date _____

Bellman Ford Algo

Time Complexity

(-) Edge = Any Edge having
(-)ve value or weight

$$O(E(V-1)) \quad \text{min(iterations)}$$

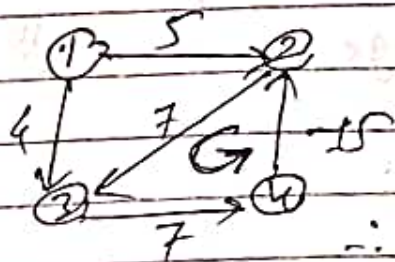
$$O(E \cdot V) \Rightarrow O(n^2) \quad \text{when } |E| = |V|$$

for complete graph = $\frac{n(n-1)}{2} \cdot (n-1)$
Bellman Ford Algo
Time complexity = $O(n^3)$

Drawback $\Rightarrow$ "

(1) Bellman Ford Algo fails for Negative cycle; But

Allow & successful for (-)ve Edges that don't form cycle



cycle (2, 3, 4) summation

$$\therefore 7 + 7 - 15 = -1$$

Algo fails here

Proof $\Rightarrow$ According to Bellman Ford Algo min(iteration) = $(n-1)$ req for shortest path.
If Any Extra iteration changes distance of any vertex then it is fails.

"Graph Traversal"

Adjacent = Neighbours

Page No. _____

Date: ____/____/____

* BFS

→ A Graph has multiple BFS Traversal sequences.

→ Data structure used = Queue

→ Pick Any vertex put on queue & put on o/p

→ while taking out from queue for o/p,

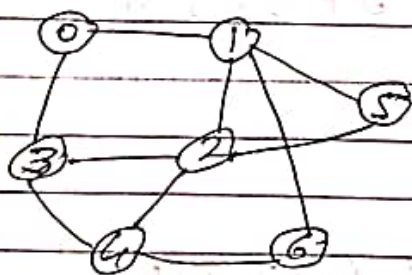
Explore all its Adjacent vertices.

→ If Any Adjacent vertex already in queue, need not to copy in queue again.

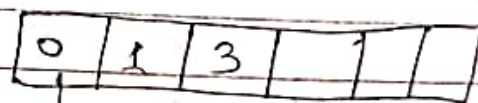
~~Graph~~ Traversing

→ Exploring of Adjacent vertices is Unordered means you can put vertex in Any order.

Ex



Queue



Result = o/p = 0

Iteration = 1: → select '0' → put on Queue.

→ put as o/p & Remove from Queue

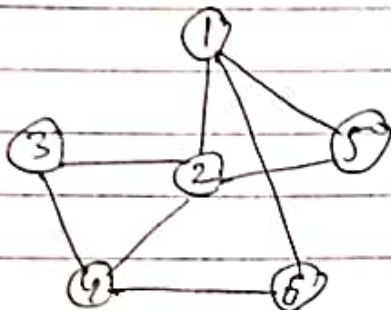
& put all its adjacent into Queue.

→ 1, 3 can be selected in any order

Iteration - 2:

Queue

0	1	3				
---	---	---	--	--	--	--



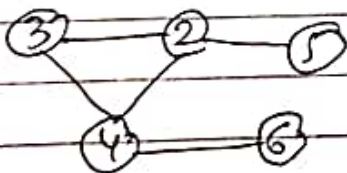
Result
o/p = 0 1

- select 1 & put at o/p & remove it from queue
- put all its adjacent vertices into queue in any order.

Iteration - 3:

Queue

0	1	3	2	5	6	
--------------	--------------	---	---	---	---	--



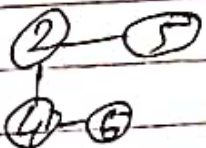
Result
o/p = 0 1 3

- select next element from queue i.e., 3
- Remove it from queue & put at o/p, bring all its adjacent into queue.

Iteration - 4:

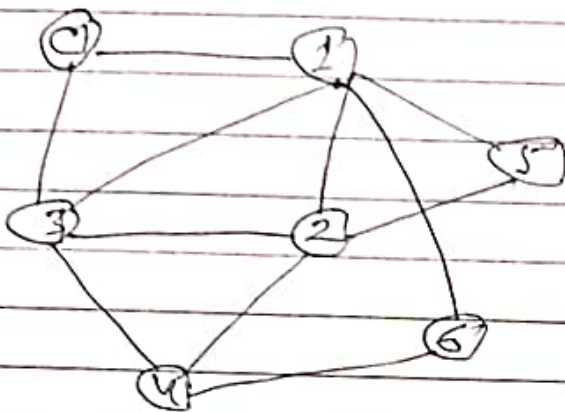
Queue

0	1	3	2	5	6	4
--------------	--------------	--------------	---	---	---	---



Result
o/p = 0 1 3 2

- select next element from queue i.e., 2
- Remove it from queue & bring all its adjacent into queue.

Ex

→ Find All possible BFS
Graph Traversal
with starting vertex '0'.

Soln

$\checkmark 0, 1, 3, 2, 5, 6, 4$
 $\checkmark 0, 3, 1, 4, 2, 5, 6$
 $\checkmark 0, 3, 1, 4, 2, 6, 5$
 ~~$0, 3, 1, 4, 5, 2, 6$~~
 ~~$0, 3, 1, 4, 5, 2, 6$~~
 ~~$0, 3, 1, 4, 6, 2, 5$~~
 ~~$0, 3, 1, 4, 6, 5, 2$~~
 $\checkmark 0, 3, 1, 2, 4, 5, 6$
 $\checkmark 0, 3, 1, 2, 4, 6, 5$

Above graph num (BFS) Traversal
possible =

(10)

BFS sequence
possible.

$3! = (2, 5, 6)$
 3 way
 $2! = (3, 4)$
 2 way

In BFS traversal
 $\text{num}(\text{Iteration}) = \text{num}(\text{Elements})$

Page No. _____
Date _____

→ If 2's Adjacents Already in queue,
no need to copy them again at queue.

Iteration - 5 :

⑤

Queue 0/1/3/2/5/6/4

④-⑥

Result
op $\Rightarrow 0\ 1\ 3\ 2\ 5$

+ select 5 which is next element at queue
after 2's deletion

→ 5's All Adjacents are removed already

Iteration - 6 :

Queue 0/1/3/2/5/6/4

④-⑥

→ remove 6 & put of Result
op $\Rightarrow 0\ 1\ 3\ 2\ 5\ 6$

Iteration - 7

④

Queue 0/1/3/2/5/6/4

→ Remove 4 &
put of op

Result
op $\Rightarrow 0\ 1\ 3\ 2\ 5\ 6\ 4$

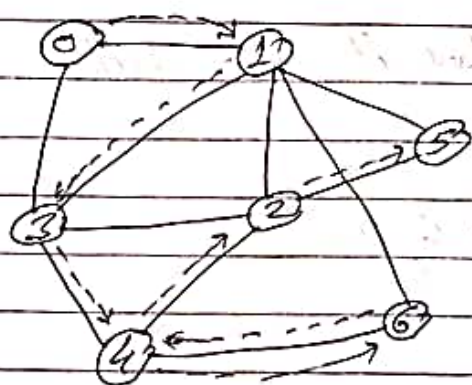
→ As no more elements left at queue

$\therefore$ op Result $\Rightarrow 0\ 1\ 3\ 2\ 5\ 6\ 4$ of BFS Traversal.

* DFS = Visit all the neighbours of current node, ^{only} select any one ^{No} make it safe if going

- Data Structure = Stack ^{pushed} _{popped}
- Multiple DFS Traversal possible for Any Graph.
- Backtrack is used to traverse remaining vertex.
- when vertex is selected put it on o/p = Result & pushed it at stack
- Go to depth of selected vertex & untill No more traversing possible from that vertex then popped that vertex & perform Backtrack.
- Vertex's Adjacent vertices can be selected in Any order.

Ex →



Stack



Result
o/p = 0

Solⁿ → select '0', put it o/p & pushed '0' on stack
Iteration - 1

Iteration=2: select 1 which is Adjacent of '0'
put it at o/p & pushed onto stack

1
0

stack

Result
o/p = 01

Iteration=3: select 3 as it is Adjacent of '1'
put it at o/p & pushed onto stack

3
1
0

stack

o/p
Result = 013

Iteration=4: select 4 as it is Adjacent to '3'
put it at o/p & pushed on stack

4
3
1
0

stack

Result
o/p $\Rightarrow$ 0134

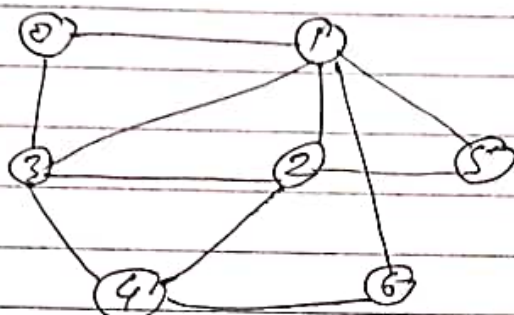
Iteration=5: select 6 as it is Adjacent to '4'
put on o/p & pushed on stack.

6
4
3
1
0

stack

Result o/p $\Rightarrow$ 01346

Ex $\Rightarrow$ Find all possible DFS Traversal of given graph.
starting with vertex '0'.



Soln

0 1 3 4 6 2 5 , 0 3 1 2 5 4 6

0 1 3 2 4 6 5 , 0 3 1 5 2 4 6

0 1 3 2 5 4 6 , 0 3 1 6 2 5

~~0 1 3 2~~ , ~~0 3 1~~

0 3 4 6 1 5 2 , 0 3 2 5 1 6 4

0 3 4 6 1 2 5 , 0 3 2 5 4 6 1 5 ... so on

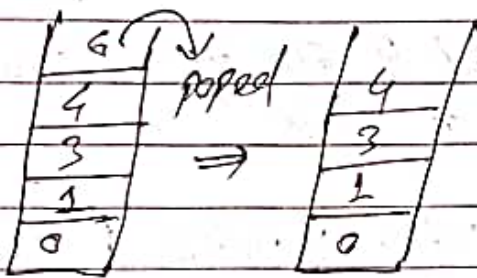
Iteration 6 :

now, perform Backtrack, as All Adjacent of 6 are already in stack &

No more depth traversal possible from 6

→ pop 6

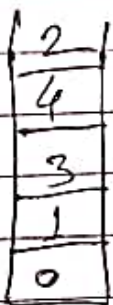
→ select 4 again



Iteration-7 : As selected '4' check all its remaining Adjacent if they are unvisited.

→ visit them, select 2, put on o/p

& ~~pop~~ pushed it into stack

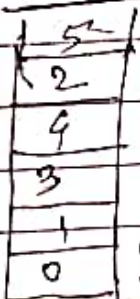


stack

Result

o/p → 0 1 3 4 6 2

Iteration-8 : → Select 5 as it is Adjacent to '2'
Put it at o/p & pushed on stack



stack

Result → 0 1 3 4 6 2 5

∴ All vertices visited, Popped All elements from stack

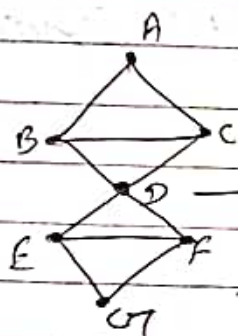
A Bi Connected Graph

Conditions

- Connected
- No Articulation point

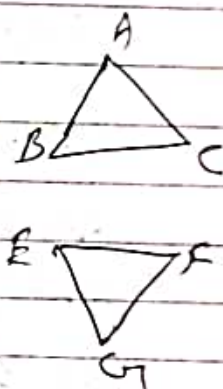
↳ whose removal disconnect graph
(Vertex & Edges incident on that vertex)

Ex

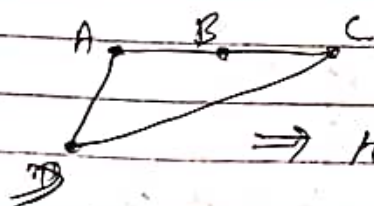


Removal of D disconnect graph

As D = Articulation point

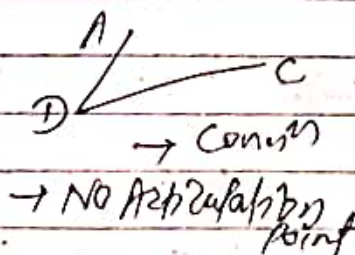


∴ Not Bi Connected graph

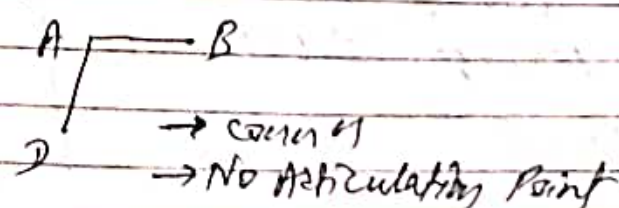


⇒ Here Removal of Any V & its incident edges will not leave / make a Remaining graph as disconnected

Remove 'B'

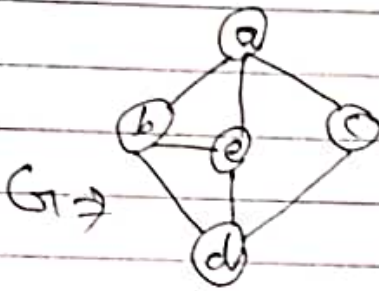


Remove 'C'



∴ This graph is Bi Connected graph.

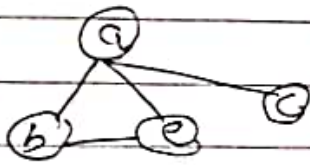
Note
⇒ If Any one vertex not satisfy condn then it is not Bi Connected graph

Ex $\Rightarrow$ 

check for Biconn graph

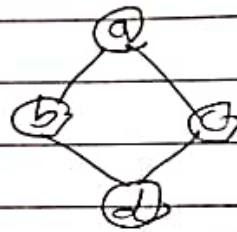
 $\rightarrow$ Remove 'e'

Remove 'd'



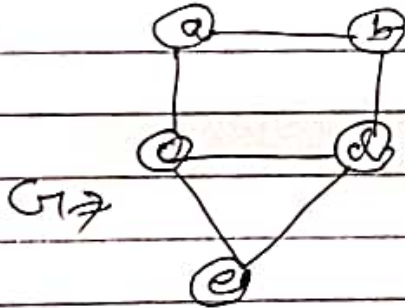
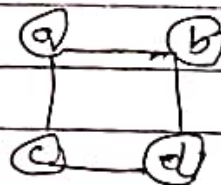
✓ conn

✓ No Articulation point

Remaining G

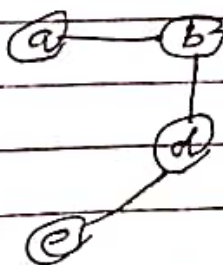
is conn

It has no Articulation point

 $\therefore$ Above $G = \text{Biconn}$ Ex $\Rightarrow$  $\rightarrow$ Remove 'e'

✓ Remaining conn

✓ No Articulation point

 $\rightarrow$ Remove 'c'

✓ Remaining conn

✓ No Articulation point

 $\therefore$ Above $G = \text{Biconn}$ graphNote $\Rightarrow$ We have to check for each vertex to decide $G = \text{Biconn}$

uses DFS to check for articulation point

Each

Tree has Articulation Points

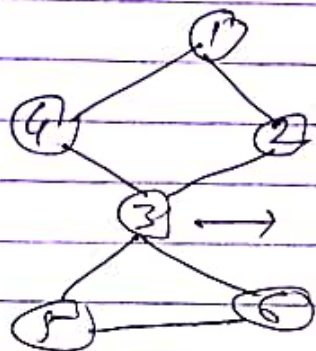
Page No. _____

Date: ____/____/____

* Articulation point $\equiv$ Any node whose removal disconnects the graph.

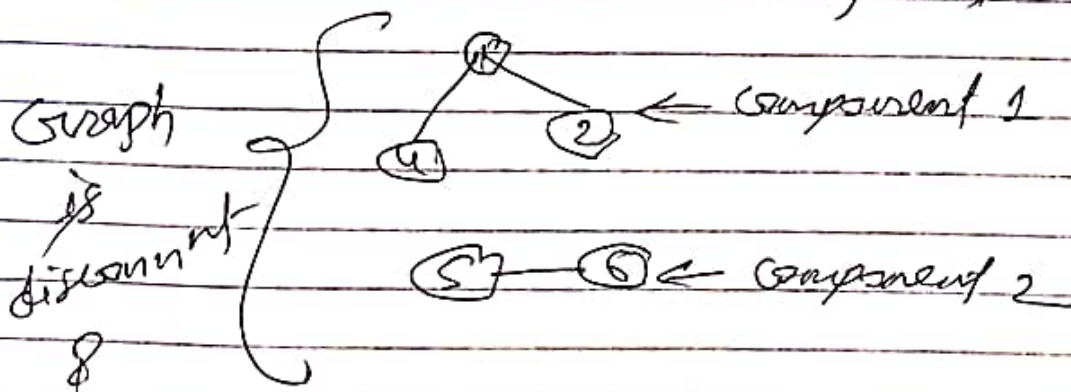
→ Whose removal disconnects the graph into different (vertex) disjoint components.

→ When vertex removed, its incident edges must be removed.



'3' vertex is articulation point

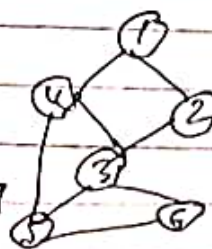
Removal of '3' leaves graph with 2 different components or disjoint graph.



has 2 components

* Remove articulation points

→ Add Edges betⁿ different components other than articulation points.



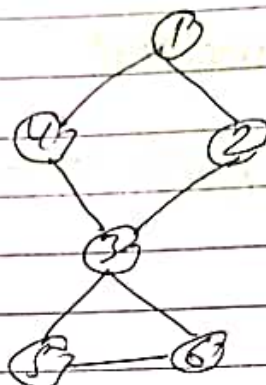
Now, G has no articulation point

for $(u, v) = (5, 3)$, $L(3) \geq dt(5)$
 $1 \geq 4 = \text{false}$ Page 3 of 3
 Date: / / point

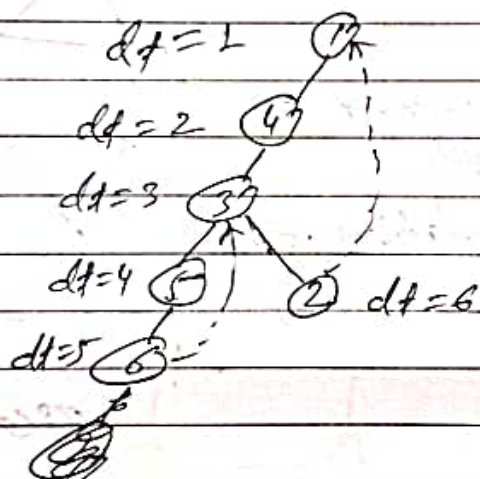
A methods to find Articulation Point of graph.

Step-I: Perform DFS on graph

DFS = 1 4 3 5 6 2



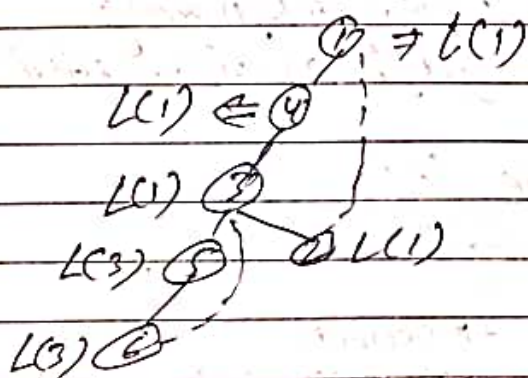
Step-II: Compute discovery time dt of DFS sequence



parent
 $u, v \leftarrow \text{child}$
 if $L(v) \geq dt(u)$
 then $u = \text{Articulation Point}$

for $(u, v) = (3, 6)$
 $L(6) \geq dt(3)$
 $3 \geq 5$
 $\therefore 3 \text{ is Ap}$

Step-III: find Level path of root/parent



DFS
 Sequence = 1 4 3 5 6 2

Vertex = 1 2 3 4 5 6

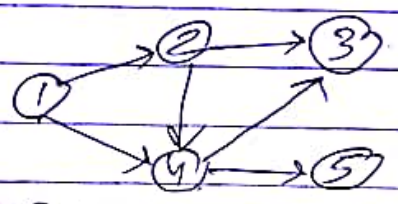
$dt(v) = 1 6 3 2 4 5$

$L(v) = 1 3 1 1 1 3$

"Topological Sort" / Topological order

- A Graph may have multiple Topological orders or sequences
- Graph should be "DAG"
 - ↳ Directed Acyclic Graph
 - Graph should not have any cycle & Graph = Directed
 - Linear ordering in such a way that for every edge "uv" there should be order $u \rightarrow v$

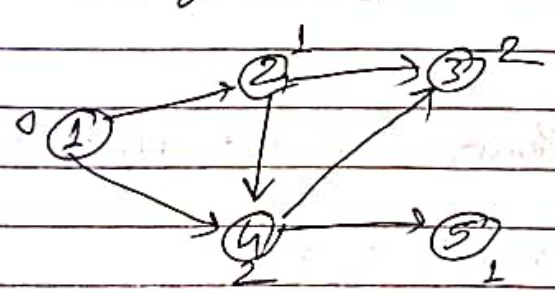
Ex →



Find Topological Sort?

Soln

- ① check wheater graph is directed / Acyclic.
- ② find Indegree of each vertex.

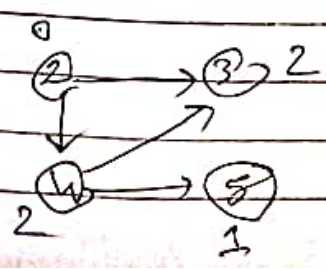


Indegree = num of edges incident on that vertex

Vertex ①

o/p ⇒ 1 vertex

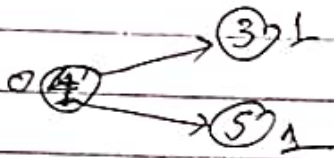
- ③ pick vertex with Indegree '0' & Remove it from graph & also remove its adjacent edges.



- ④ Again find Indegree of each vertex

Vertex 2

- ⑤ pick vertex with Indegree '0' & put at o/p
remove it & it's Adjacent edges
(Incident)

o/p $\Rightarrow 1, 2$

- ⑥ Find Indegree of each vertex:

- ⑦ pick vertex with Indegree '0', put at o/p

Vertex 4 & Remove it & it's Incident edges.

③⁰⑤⁰o/p $\Rightarrow 1, 2, 4$

- ⑧ If Remaining vertex / ^{at} Any time more than 1 vertex
has Indegree '0' then we can pick Any vertex
~~to~~ among them

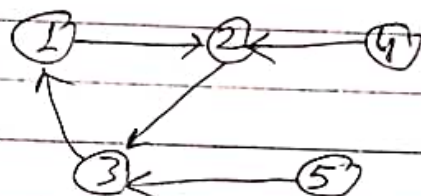
Vertex 3⑤⁰o/p $\Rightarrow 1, 2, 4, 3$

- ⑨ The last vertex also added at o/p $\Rightarrow 1, 2, 4, 3, 5$

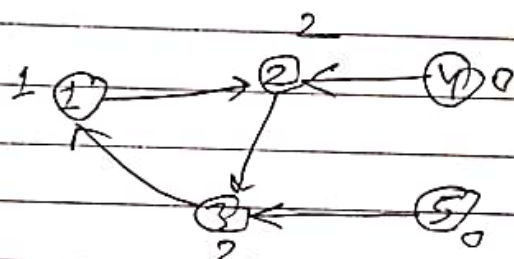
Another Topological

sort for this graph = 1, 2, 4, 5, 3

Ex $\Rightarrow$ Find Topological sorting of given graph.

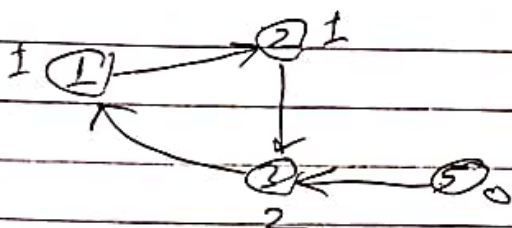


Solⁿ: $\Rightarrow$ Find Indegree of all vertex.



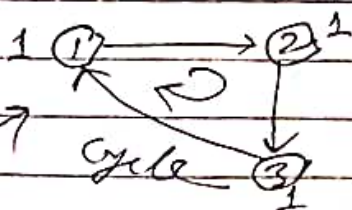
$\rightarrow$ check for Graph
Directed Acyclic

$\rightarrow$ pick vertex 4 as Indegree = 0 & put it at o/p & remove its incident edges



O/P $\Rightarrow$ 4

$\rightarrow$ pick vertex 5 as Indegree = 0 & put it at o/p & remove its adjacent edges



O/P $\Rightarrow$ 4, 5

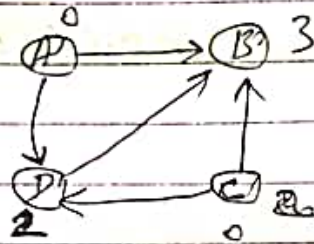
Proof $\rightarrow$

cycle

Here, so topological sorting is not possible

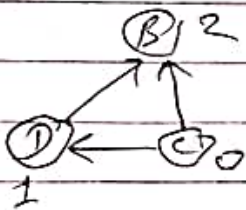
As now, no vertex with indegree 0,

Ex: Find Topological sort of given graph



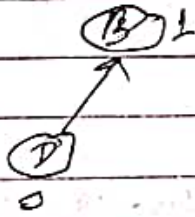
Soln $\Rightarrow$ check for directed & Acyclic $\checkmark$

$\rightarrow$ pick (A) in-degree = 0, remove all its adjacent edges



O/P $\Rightarrow$ A

$\rightarrow$ pick (C) in-degree = 0, remove all its adjacent edges



O/P $\Rightarrow$ A, C

$\rightarrow$ pick (D) in-degree = 0, remove all its adjacent edges



O/P $\Rightarrow$ A, C, D,

$\rightarrow$ pick (B) as it is last node

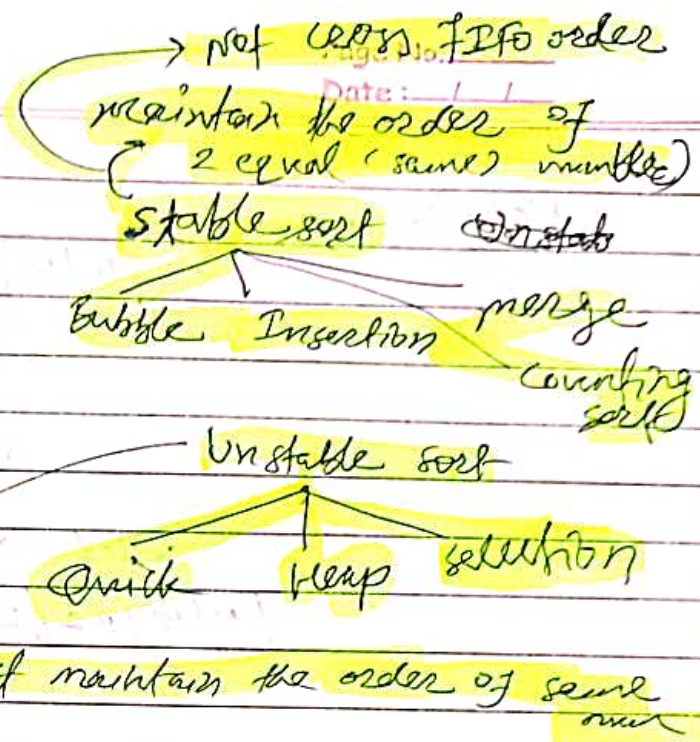
O/P $\Rightarrow$ A, C, D, B $\checkmark$

other Topological
Sequence

is also possible $\Rightarrow$ A, C, D, B $\checkmark$

Sorting

- ① Bubble sort
- ② selection sort
- ③ Insertion sort
- ④ Quick sort
- ⑤ Heap sort
- ⑥ Radix sort



① Bubble sort → fixed the largest element at last location
then 2nd largest at 2nd last location
likewise

Ascending → compare adjacent elements,
→ check element 1 < element 2 if yes
→ then swap them
→ consider next pair of elements for comparison.
do it till (n-1) last element reached.
→ num (iteration) reqd (n-1), where n = total num of elements.

Ex ⇒ 5, 1, 4, 2, 8, 9

Iteration-I : [5 | 1 | 4 | 2 | 8 | 9]

↑↑ check (<) if condn violates then swap ⇒ [1 | 5 | 4 | 2 | 8 | 9]

Iteration-II :

[1 | 5 | 4 | 2 | 8 | 9]
↑↑

[2 | 4 | 5 | 2 | 8 | 9]
check (<) if condn violates then swap.

Inplace = Bubble / select / ~~Insert~~ / Heap / shell
 Insertion
Not Inplace = merge / Radix / Bucket

Page No. _____
 Date ____/____/____

Iteration - III $\Rightarrow$

1	4	5	2	8	9
---	---	---	---	---	---

 $\uparrow \uparrow$
 check & swap $\times$ $\Rightarrow$

1	4	2	5	8	9
---	---	---	---	---	---

Iteration - IV $\Rightarrow$

1	4	2	5	8	9
---	---	---	---	---	---

 $\uparrow \uparrow$
 check ($<$)
 swap $\times$ $\Rightarrow$

1	4	2	5	8	9
---	---	---	---	---	---

 $\uparrow \uparrow$
 check ($<$)
 swap $\times$

Iteration - V

1	4	2	5	8	9
---	---	---	---	---	---

 $\uparrow \uparrow \uparrow \uparrow$
 check $\checkmark$ swap $\times$ check ($<$) $\times$ swap $\checkmark$

1	2	4	5	8	9
---	---	---	---	---	---

 $\uparrow \uparrow \uparrow \uparrow$
 check $\checkmark$ swap $\checkmark$

$\rightarrow$ After $(n-1)$ Iterations we will get sorted Array

$\rightarrow$ This is Bubble sort in Ascending order

code

```

Bubble sort(int Arr[], int N)
{
  for (i=0; i < N-1; i++)
    for (j=0; j < N-i-1; j++)
      { if (Arr[j] < Arr[j+1])
          No swap, just consider Next pair for comp
        else
          swap Arr[j+1] = Arr[j]
            Arr[j] = Arr[j+1]
      }
}
  
```

② selection sort → Unstable sort

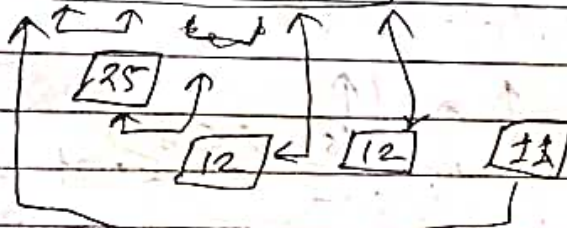
① → selection of smallest element in the remaining array

② → swapping: bring it to the starting position.

Ex ⇒ 64, 25, 12, 22, 11

Unsorted position

Iteration-I ⇒ 64 | 25 | 12 | 22 | 11



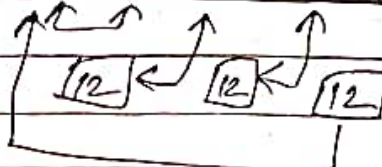
checking/finding smallest element

Bring it to Beginning

Swapping with 1st position

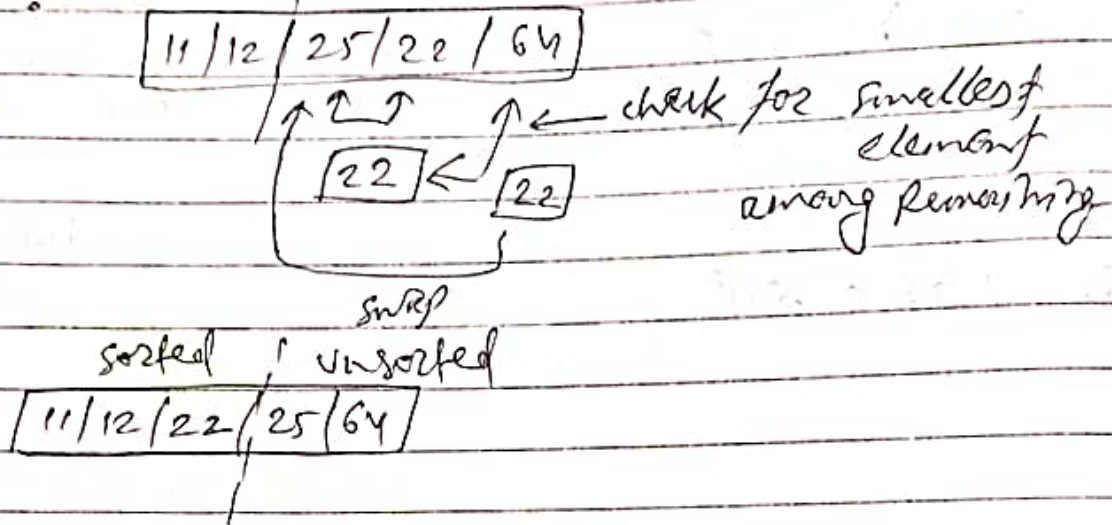
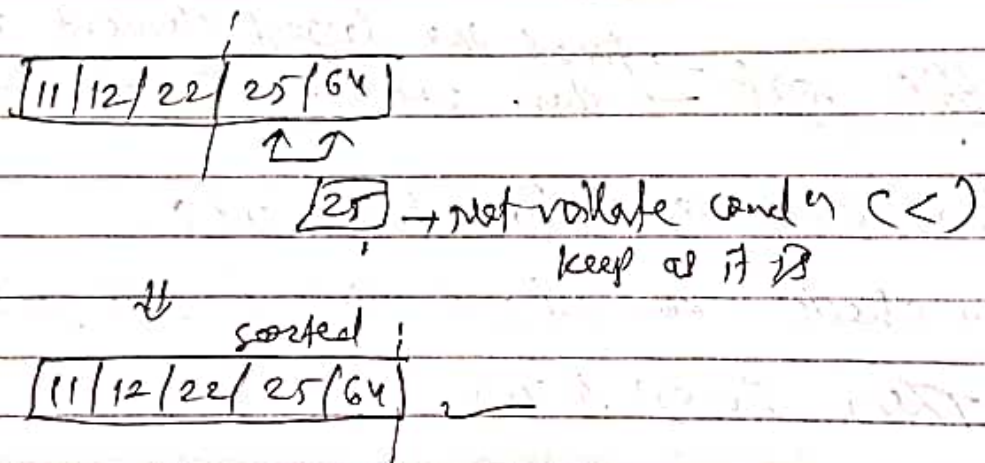
sorted : unsorted position
position | 11 | 25 | 12 | 22 | 64

Iteration-II ⇒ 11 | 25 | 12 | 22 | 64



swap with 2nd position

sorted : unsorted
| 11 | 12 | 25 | 22 | 64

Operation $\rightarrow$ III :Operation $\rightarrow$ IV :

* Insertion sort

- Find the correct position of elements by checking $cond^n (<)$ should violate.
- ~~swap~~ Check that element with remaining all elements at sorted portion.
- if $cond^n$ is not violating offer some x^n then swap it.

Ex → 7, 8, 5, 2, 4, 6, 3

Iteration - I:

sorted / unsorted
[7 | 8 | 5 | 2 | 4 | 6 | 3]

↑

[8] $7 < 8 \Rightarrow$ Not violate, move to next element

↑
[5] $7 < 5$ violate, insert 5 at Beginning of 7, But shift 8 first then 7

sorted / unsorted
[5 | 7 | 8 | 2 | 4 | 6 | 3]

Iteration - II:

[5 | 7 | 8 | 2 | 4 | 6 | 3]

↑

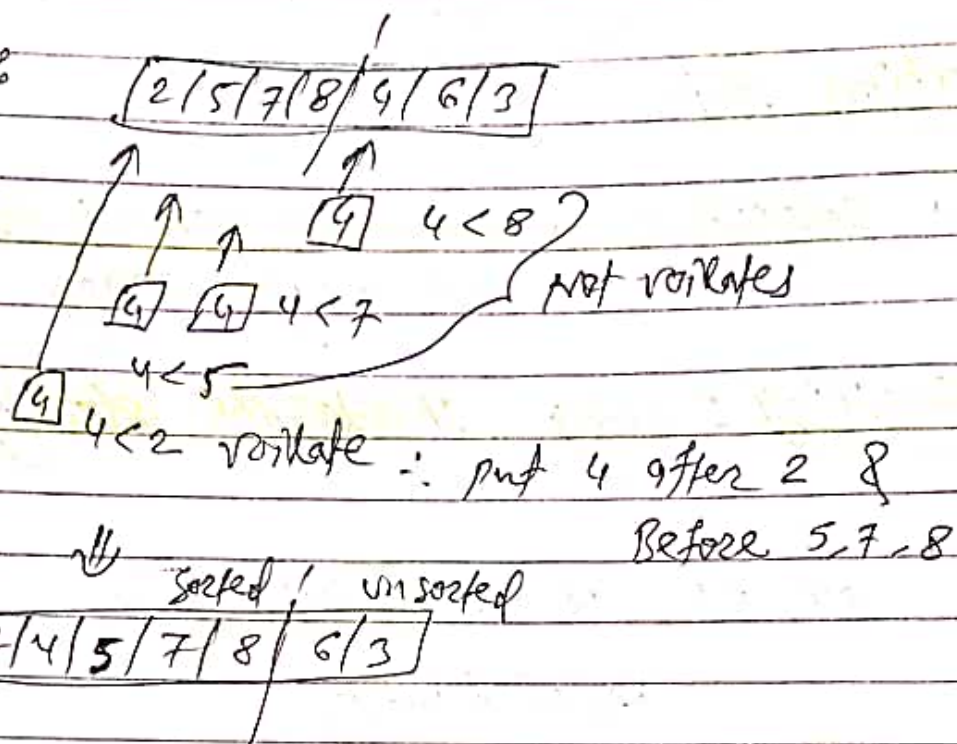
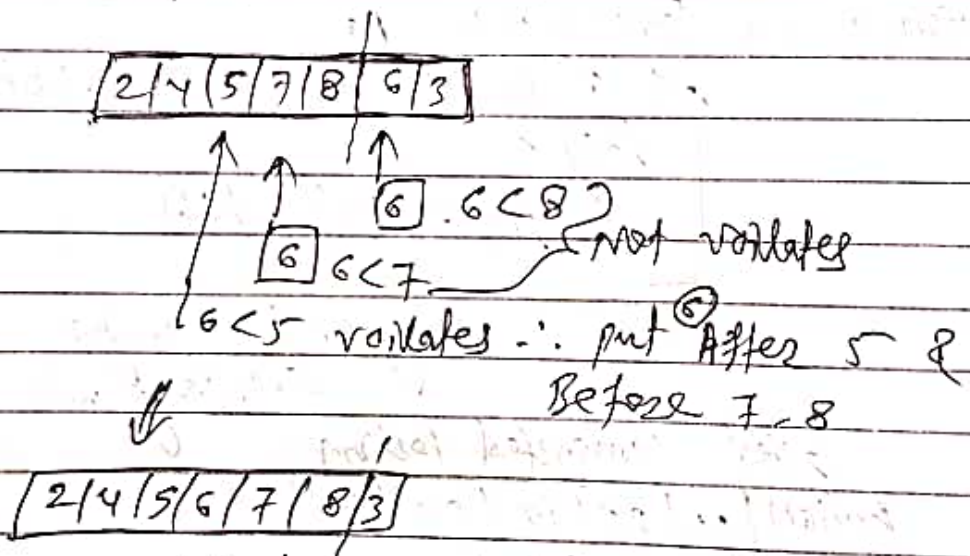
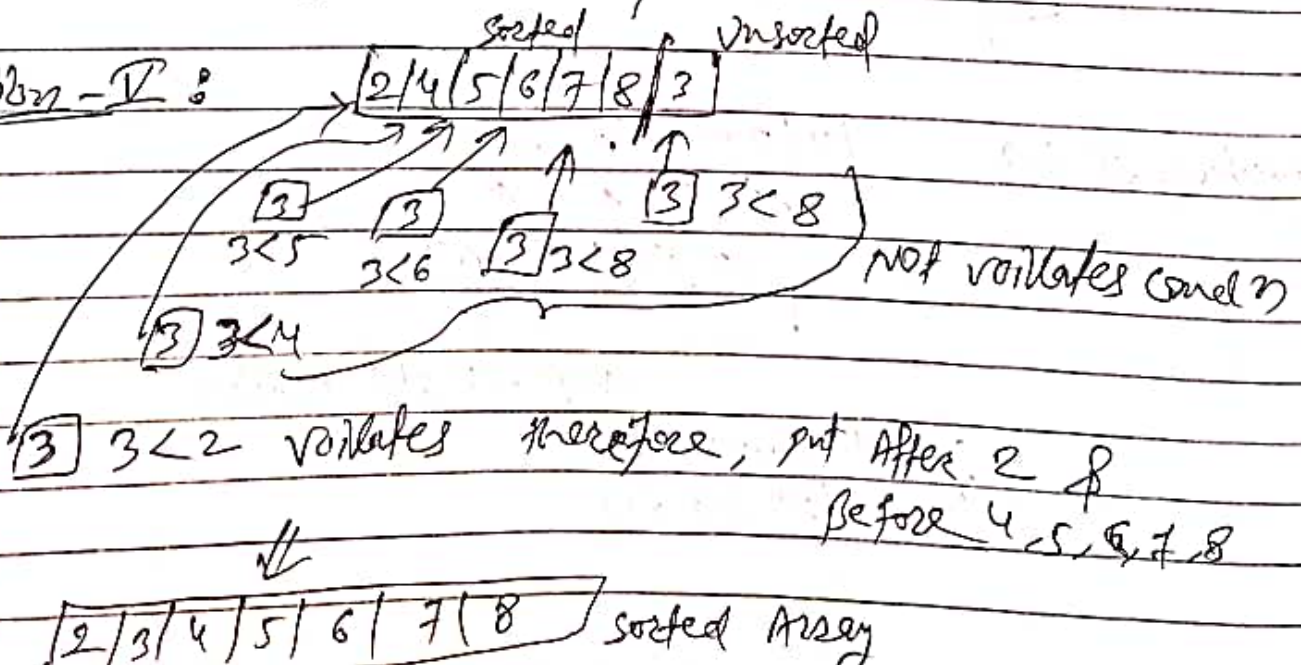
↑ [2] ~~8~~ $2 < 8 \Rightarrow$ Not violate

↑ [2] $2 < 7$ not violate

[2] < 5 not violate

Put 2 at Beginning

sorted / unsorted
[2 | 5 | 7 | 8 | 4 | 6 | 3]

Iteration - III :Iteration - IV :Iteration - V :

* Radix Sort → It uses Counting Sort Algorithms

↳ It sort only Numbers

↳ Sort numbers from LSB to MSB of digit

Ex ⇒ 170, 45, 75, 90, 802, 24, 2, 66

Iteration-I : Consider ^{one's} ~~ten's~~ place

[170 | 45 | 75 | 90 | 802 | 24 | 2 | 66]

⇓

[170 | 90 | 802 | 2 | 24 | 45 | 75 | 66]

LSB with least number comes at Beginning

Note ⇒ 170 comes before 90 & 802 comes before 2 as } They

occured before them in i/p string

Iteration-II : Consider ten's place

[170 | 90 | 802 | 2 | 24 | 45 | 75 | 66]

↑ not have to ten's place

⇓

∴ consider 02

[802 | 2 | 24 | 45 | 66 | 170 | 75 | 90]

Iteration - III :hundreds
Consider 100's place

802	2	24	45	66	170	75	90
x	x	x	x		x	x	

Those numbers not have hundreds place, Assume "00" and "0" in front of them

↓

2	24	45	66	75	90	170	802
---	----	----	----	----	----	-----	-----

→ sorted Array

Note ⇒ Num (Iteration)

Rgs to srf = size of digits of

Numbers

10 place numbers srf = 2 iterations
 100 place num srf = 3 iterations
 1 place num srf = 1 iterations

Radix sort
 Time Complexity

= $O(n \times d)$

(or)

$O(d(n \times B))$

$n = \text{num (elements)}$
 $d = \text{num (digits)}$
 $B = \text{base or bucket size}$

$B = \text{base or bucket size}$

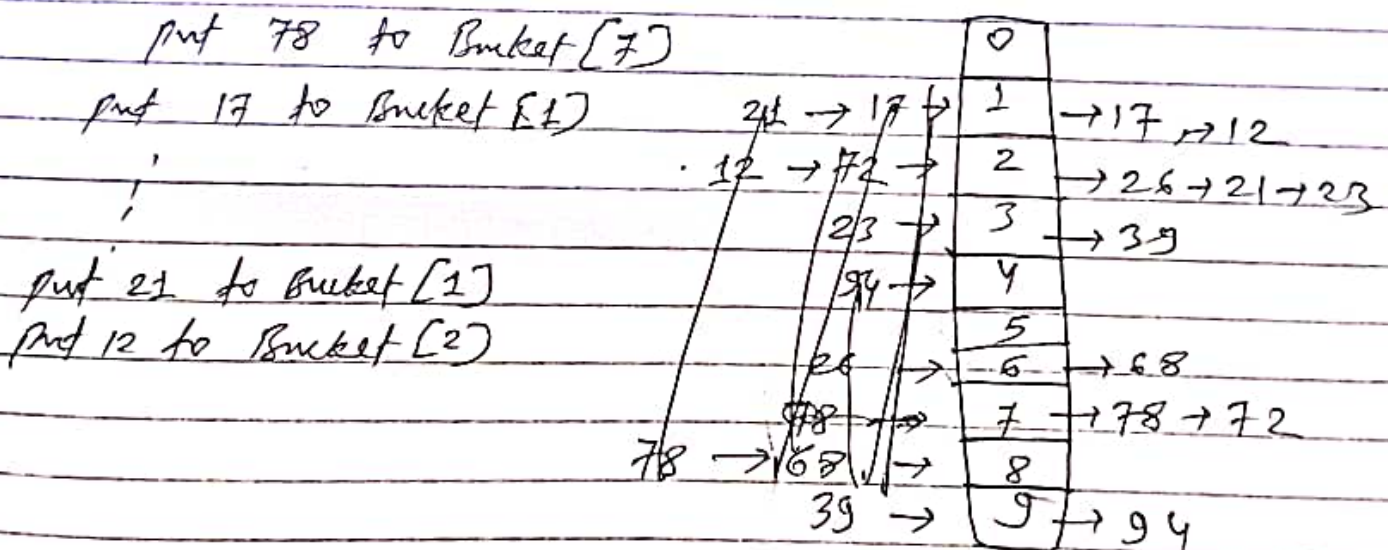
* Bucket sort

- Buckets are created to put elements into them
- It uses Insertion Sort Algo to sort elements in each Bucket.

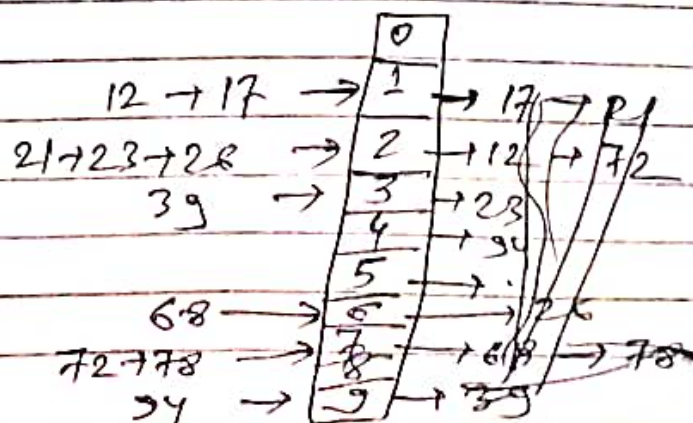
Ex →

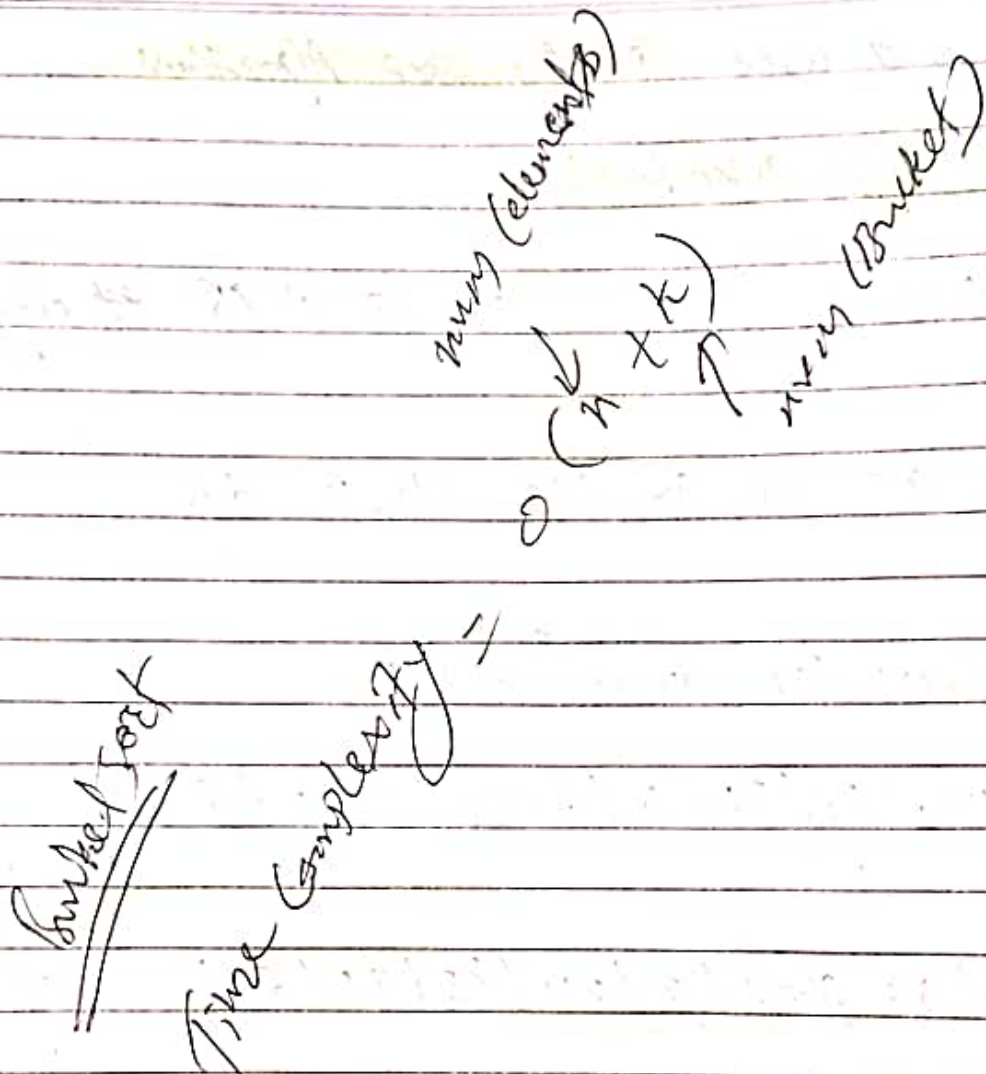
78, 17, 39, 26, 72, 94, 21, 12, 23, 68

Iteration-I ⇒ create Bucket 0 to 9



Iteration-II ⇒ Arrange them within Bucket using Insertion Sort





Step 2: Print all buckets in increasing order

O/P $\Rightarrow$ 12, 17, 21, 23, 28, 39, 68, 72, 78, 94